

Experiment No.	3
Experiment Title	Regression Analysis using Linear and Regularized Models
Student Name	Mohamed Farih
Register Number	3122247001035
GitHub Repository	https://github.com/farih07/Machine-Learning-Lab
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	August 16, 2026

1 Objective

The objective of this experiment is to implement Linear Regression along with three regularized regression variants — Ridge, Lasso, and Elastic Net — to predict a continuous target variable from the loan-application dataset, to tune each regularized model’s hyperparameters using 5-fold cross-validated Grid Search, to evaluate and compare all four models using MAE, MSE, RMSE, R^2 , and training time, and to analyze the resulting overfitting/underfitting behaviour and bias–variance trade-offs.

2 Problem Statement

Input: 30,000 loan-application records, each described by 23 features spanning applicant demographics (Age, Gender, Income), employment information (Profession, Type of Employment, Income Stability), loan/property details (Current Loan Expenses, Property Price, Property Age, Property Type), and credit history (Credit Score, No. of Defaults, Has Active Credit Card).

Output: A predicted continuous value for the target variable **Loan Amount Request (USD)**, produced by each of the four trained regression models (Linear, Ridge, Lasso, Elastic Net).

Objective: To build accurate regression models that predict the requested loan amount from applicant and property features, to control model complexity through regularization, and to empirically compare the trained models on standard regression metrics and generalization behaviour, in order to identify the model best suited for this prediction task.

3 Dataset Description

Dataset Name	Predict Loan Amount Data
Dataset Source	Kaggle (https://www.kaggle.com/datasets) — “Predict Loan Amount Data”
Number of Samples	30,000 (raw); 30,000 retained after pre-processing (no rows dropped — missing values were imputed rather than removed)
Number of Features	23 features after removing the target column (11 categorical, 12 numerical)

Target Variable	Loan Amount Request (USD) — automatically detected by the notebook’s target-column search logic (see Section 6 for a note on this detection)
Missing Values	Present in 11 of 24 original columns (e.g. Income (USD): 4576, Type of Employment: 7270, Property Age: 4850, Dependents: 2493, Credit Score: 1703, Has Active Credit Card: 1566, Income Stability: 1683, Current Loan Expenses (USD): 172, Property Location: 356, Loan Sanction Amount (USD): 340, Gender: 53); 0 missing values remained after imputation
Train–Test Split	80% train (24,000 samples) / 20% test (6,000 samples), <code>random_state=42</code> (no stratification, as this is a regression task)

4 Exploratory Data Analysis

Exploratory Data Analysis was performed on the scaled feature set (combined with the target variable) to understand feature distributions, the target distribution, and correlations between features and the target.

- Dataset overview: 30,000 loan applications, 23 features (11 categorical, 12 numerical) plus the target column.
- Statistical summary: computed via `eda_df.describe()` on the standardized feature set combined with the target.
- Missing value analysis: 11 of 24 raw columns contained missing values (see Section 3); all were resolved during preprocessing (Section 6).
- Correlation matrix: computed for all standardized features together with the target variable, and the per-feature correlation with the target was additionally sorted and printed.
- Feature distribution: examined via a histogram of the target variable and scatter plots of the four most target-correlated features.

Correlation Heatmap

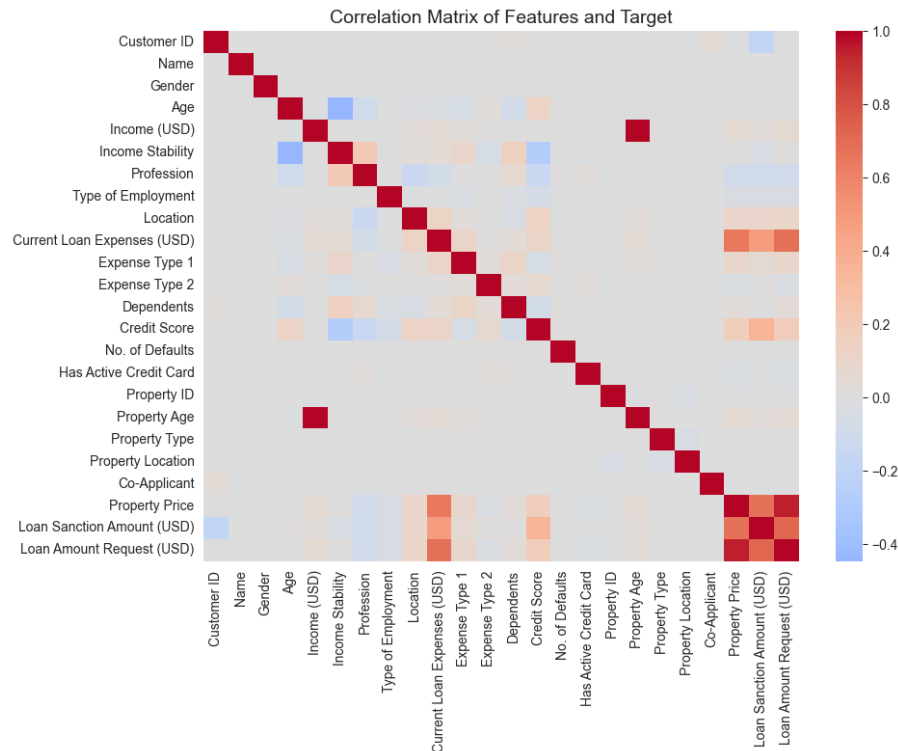


Figure 1: Correlation heatmap of all standardized features together with the target variable, Loan Amount Request (USD).

Inference: The correlation analysis (printed alongside the heatmap) shows that **Property Price** ($r = 0.9484$), **Loan Sanction Amount (USD)** ($r = 0.7212$), and **Current Loan Expenses (USD)** ($r = 0.6805$) are by far the most strongly correlated features with the target, while most remaining features (Credit Score, Income, Property Age, Dependents, and the demographic/categorical fields) show weak correlation ($-r < 0.20$). This indicates that the loan amount requested is driven predominantly by a small number of monetary features that are naturally proportional to it, with the remaining applicant/property attributes contributing comparatively little linear signal.

Target Variable Distribution

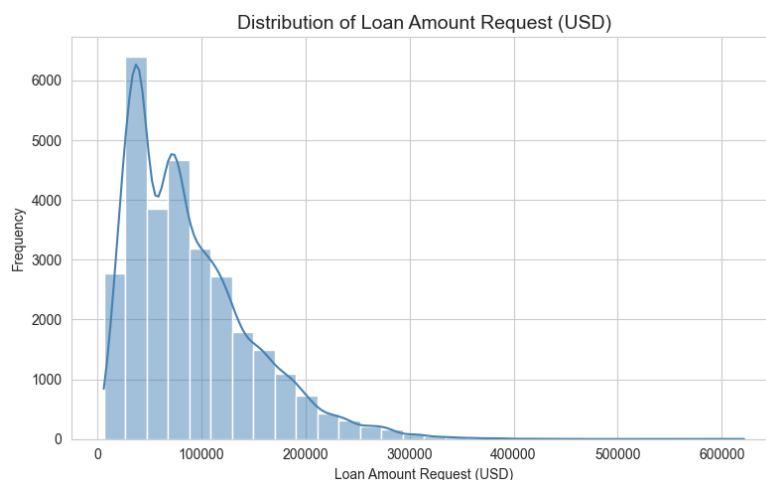


Figure 2: Distribution of the target variable, `Loan Amount Request (USD)`, with a kernel density estimate overlay.

Inference: The target variable has a mean of \$88,826.33, a median of \$75,128.07, and a standard deviation of \$59,536.95. Since the mean is noticeably higher than the median, the distribution is right-skewed — most loan requests cluster at lower-to-moderate amounts, with a long tail of larger requested amounts. This skew is worth noting when interpreting MAE/RMSE, since large-value outliers in the tail will disproportionately affect squared-error-based metrics (MSE/RMSE) relative to MAE.

Feature vs. Target Scatter Plots



Figure 3: Scatter plots of the four features most correlated with the target (Property Price, Loan Sanction Amount (USD), Current Loan Expenses (USD), Credit Score) against Loan Amount Request (USD).

Inference: Property Price shows the clearest, tightest linear relationship with the target, consistent with its very high correlation coefficient. Loan Sanction Amount (USD) and Current Loan Expenses (USD) also show visible upward trends, though with more scatter/noise around the trend line. Credit Score shows only a weak, diffuse relationship with the target, confirming that creditworthiness alone is a comparatively weak linear predictor of the requested loan amount in this dataset.

5 Data Preprocessing

- **Target column detection:** The notebook automatically searches for a target column by first checking a list of likely exact names, then falling back to the first column whose name contains the substring “amount”. Under this logic, **Loan Amount Request (USD)** was detected and used as the target column, since it is the first “amount”-containing column in the dataset’s column order — rather than **Loan Sanction Amount (USD)**, which the assignment brief refers to as “the loan amount sanctioned”. This is a characteristic of the automatic detection logic as implemented, and is noted here for completeness; it also means **Loan Sanction Amount (USD)** was instead retained as an input feature (see below).
- **ID column removal:** The notebook attempts to drop identifier columns whose name is exactly “id” or ends with “_id” (underscore-id). Because the dataset’s

identifier columns are named `Customer ID` and `Property ID` (with a space rather than an underscore before “ID”), this pattern did not match, and both columns were retained as features. `Customer ID` was label-encoded (categorical) and `Property ID` was kept as a numeric feature. Their correlation with the target was subsequently found to be negligible ($r = 0.0021$ and $r = -0.0084$ respectively; see Section 5), so this had minimal practical effect on model performance, though it is not ideal preprocessing practice.

- **Missing value handling:** Missing numerical values were imputed with the median of their respective column; missing categorical values were imputed with the mode (most frequent value) of their respective column. This resolved all missing values (0 remaining after cleaning), rather than dropping any of the 30,000 rows.
- **Encoding:** All 11 categorical columns (`Customer ID`, `Name`, `Gender`, `Income Stability`, `Profession`, `Type of Employment`, `Location`, `Expense Type 1`, `Expense Type 2`, `Has Active Credit Card`, `Property Location`) were converted to numeric codes using `LabelEncoder`.
- **Feature scaling:** All 23 features were standardized using `StandardScaler` (zero mean, unit variance) prior to modelling, which is particularly important for Ridge, Lasso, and Elastic Net, since their regularization penalties are applied directly to the coefficient magnitudes and are only meaningful when features share a common scale.
- **Train–test split:** An 80/20 split (`random_state=42`) was used, producing 24,000 training samples and 6,000 test samples.
- **Feature engineering:** No additional feature engineering (e.g. polynomial or interaction terms) was performed; the 23 preprocessed columns were used directly as model inputs.

6 Machine Learning Algorithm

Linear Regression

Working principle: Linear Regression models the target as a weighted linear combination of the input features. Given feature vector $x \in \mathbb{R}^d$, it predicts $\hat{y} = w_0 + \sum_{i=1}^d w_i x_i$, with weights w chosen to minimize the sum of squared residuals on the training data.

Mathematical formulation:

$$\hat{w} = \arg \min_w \|y - Xw\|_2^2 \quad (1)$$

Advantages: simple, fast to train (0.0131s in this experiment), highly interpretable coefficients, no hyperparameters to tune.

Limitations: sensitive to multicollinearity among features, has no built-in mechanism to prevent overfitting on noisy or highly correlated features, and can produce unstable/large coefficients when features are correlated (e.g. `Income (USD)`’s coefficient changed sign between Linear Regression and the regularized models in this experiment, indicating some coefficient instability — see Section 13).

Ridge Regression

Working principle: Ridge Regression adds an L_2 penalty on the coefficient magnitudes to the ordinary least-squares objective, shrinking coefficients toward zero (but not exactly to zero) to reduce variance.

Mathematical formulation:

$$\hat{w} = \arg \min_w \|y - Xw\|_2^2 + \alpha \|w\|_2^2 \quad (2)$$

Advantages: reduces overfitting and coefficient instability caused by multicollinearity, keeps all features in the model, closed-form and fast to fit (0.0075s with the tuned α).

Limitations: does not perform feature selection (no coefficients are driven to exactly zero), and requires tuning the regularization strength α .

Lasso Regression

Working principle: Lasso Regression adds an L_1 penalty on the coefficient magnitudes, which can drive some coefficients to exactly zero, effectively performing feature selection.

Mathematical formulation:

$$\hat{w} = \arg \min_w \|y - Xw\|_2^2 + \alpha \|w\|_1 \quad (3)$$

Advantages: produces sparse, more interpretable models by eliminating weakly-informative features; achieved the best test R^2 (0.9043) among all four models in this experiment.

Limitations: slower to fit via coordinate descent (1.5978s, over 100× slower than Ridge in this experiment), can behave erratically when features are highly correlated (arbitrarily selecting one of several correlated features), and requires tuning α .

Elastic Net Regression

Working principle: Elastic Net combines the L_1 and L_2 penalties of Lasso and Ridge, controlled by a mixing parameter `l1_ratio`, balancing feature selection (from Lasso) with the stability of grouped shrinkage (from Ridge).

Mathematical formulation:

$$\hat{w} = \arg \min_w \|y - Xw\|_2^2 + \alpha \rho \|w\|_1 + \frac{\alpha(1 - \rho)}{2} \|w\|_2^2 \quad (4)$$

where ρ is the `l1_ratio`.

Advantages: more robust than pure Lasso when features are correlated, still allows partial sparsity; the tuned model in this experiment selected a high `l1_ratio` of 0.8, favouring Lasso-like behaviour.

Limitations: introduces a second hyperparameter (`l1_ratio`) to tune alongside α , increasing the hyperparameter search space, and in this experiment did not outperform plain Lasso (test $R^2 = 0.9008$ vs. 0.9043).

7 Experimental Setup

Python Version	3.x
Scikit-Learn Version	1.x (latest stable)
IDE	Jupyter Notebook
Operating System	Windows (local machine)
Hardware	Standard CPU (no GPU required)

8 Hyperparameter Selection

Each regularized model was tuned using `GridSearchCV` with 5-fold `KFold` cross-validation (`random_state=42`), scored on R^2 , over the search spaces specified in the lab manual.

Parameter	Value
Ridge α search space	0.01, 0.1, 1, 10, 100
Lasso α search space	0.001, 0.01, 0.1, 1, 10
Elastic Net α search space	0.01, 0.1, 1, 10
Elastic Net <code>l1_ratio</code> search space	0.2, 0.5, 0.8
Cross-validation folds	5 (<code>KFold</code> , shuffled, <code>random_state=42</code>)
Ridge best parameters	$\alpha = 10$
Lasso best parameters	$\alpha = 10$
Elastic Net best parameters	$\alpha = 0.01$, <code>l1_ratio</code> = 0.8

Table 1: Hyperparameter Tuning Summary

Model	Search Method	Best Parameters	Best CV R^2
Ridge Regression	Grid Search	$\alpha = 10$	0.9165
Lasso Regression	Grid Search	$\alpha = 10$	0.9165
Elastic Net Regression	Grid Search	$\alpha = 0.01$, <code>l1_ratio</code> = 0.8	0.9165

Note: Linear Regression has no tunable hyperparameters and was therefore not included in the Grid Search process; it was fit directly on the training set (Section 13).

9 Results

The table below reports the held-out test-set performance of the best-performing model overall (Lasso Regression, selected by R^2).

Metric	Value
MAE	11346.2351
MSE	334531858.31
RMSE	18290.2121
R^2	0.9043
Training Time	1.5978 s

10 Comparison Tables

Table 2: Cross-Validation Performance ($K = 5$)

Model	MAE	MSE	RMSE	R^2
Linear Regression	Not applicable — no hyperparameters to cross-validate			
Ridge Regression	Not tracked separately in the notebook			0.9165
Lasso Regression	Not tracked separately in the notebook			0.9165
Elastic Net Regression	Not tracked separately in the notebook			0.9165

Note: the notebook's GridSearchCV was scored on R^2 only, so only the best cross-validated R^2 is available for each regularized model (reproduced from Table 1). MAE, MSE, and RMSE were computed only on the final held-out test set (Table 3), not separately under cross-validation, and are reported there instead.

Table 3: Test Set Performance

Model	MAE	MSE	RMSE	R^2	Train Time
Lasso Regression	11346.2351	3.3453e+08	18290.2121	0.9043	1.5978 s
Elastic Net Regression	11365.7532	3.4670e+08	18619.8415	0.9008	0.5837 s
Ridge Regression	11358.9479	3.4848e+08	18667.4966	0.9003	0.0075 s
Linear Regression	11357.0900	3.4913e+08	18685.1014	0.9001	0.0131 s

Sorted by R^2 score (best first). Lasso Regression is the best-performing model overall on the held-out test set, though all four models perform very similarly (R^2 between 0.900 and 0.904), reflecting the dominant influence of the same small set of strongly correlated features (Property Price, Loan Sanction Amount, Current Loan Expenses) across all four models.

Table 4: Coefficient Comparison

Feature	Linear	Ridge	Lasso	Elastic Net
Income (USD)	−860.06	1227.48	0.00	1593.73
Current Loan Expenses (USD)	6170.82	6186.49	6180.18	6247.89
Property Age	4361.36	2255.68	3060.28	1840.02
Property Price	46531.36	46489.78	46544.02	46333.11
Loan Sanction Amount (USD)	8853.40	8870.21	8834.71	8935.76
Credit Score	−1263.42	−1263.34	−1247.03	−1264.60

*A representative subset of the full 23-feature coefficient table is shown above (the target-correlated and most-shrunk features); the complete coefficient table for all 23 features was generated in the notebook. Notably, **Income (USD)** is the only feature Lasso reduced to exactly zero, and it is also the feature whose sign is least stable across models (negative under Linear Regression, positive under Ridge and Elastic Net) — see Section 14 for further discussion.*

11 Result Visualization

Include all graphs generated during the experiment, with figure, caption, and inference for each.

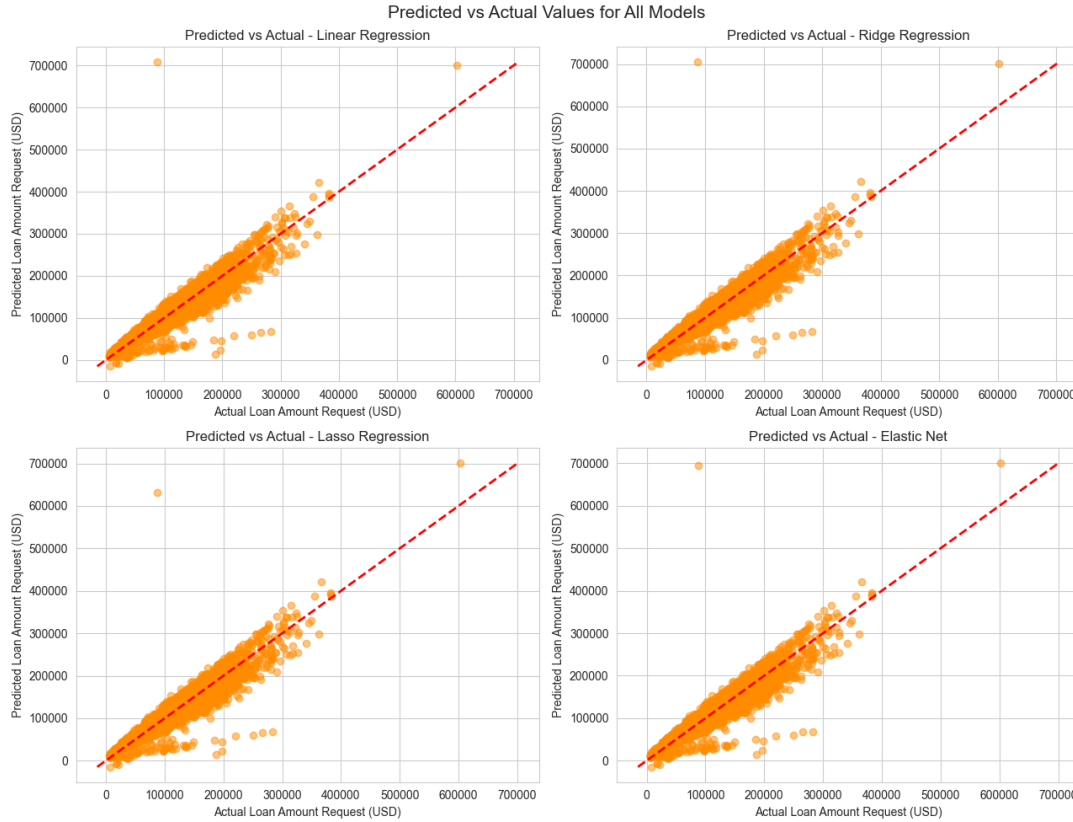


Figure 4: Predicted vs. Actual values on the test set for all four models.

Inference: All four models show points clustering closely around the diagonal $y = \hat{y}$ reference line for small-to-moderate loan amounts, confirming the high R^2 scores (0.90+) reported in Table 3. All four models show visibly increasing scatter/spread away from the diagonal at larger actual loan amounts, indicating that prediction error grows with the size of the requested loan — consistent with the right-skewed target distribution observed in Section 5. The four subplots are visually very similar, reflecting how close the four models' performance is overall.

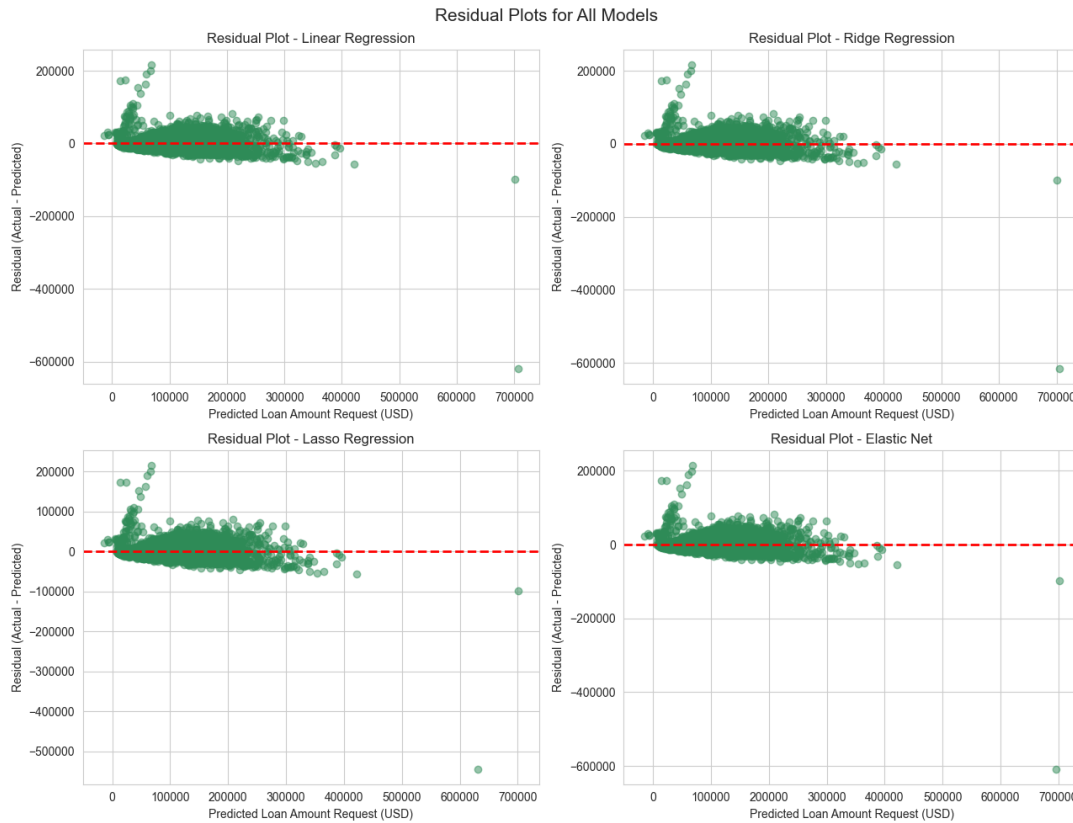


Figure 5: Residual plots (Actual – Predicted vs. Predicted) on the test set for all four models.

Inference: Residuals are roughly centered around zero for low-to-moderate predicted values, but the spread of residuals widens noticeably as predicted loan amount increases, indicating heteroscedasticity (non-constant error variance) rather than a purely random, constant-variance error pattern. This pattern is nearly identical across all four models, suggesting the increasing residual spread is a property of the data/target distribution rather than something regularization can correct.

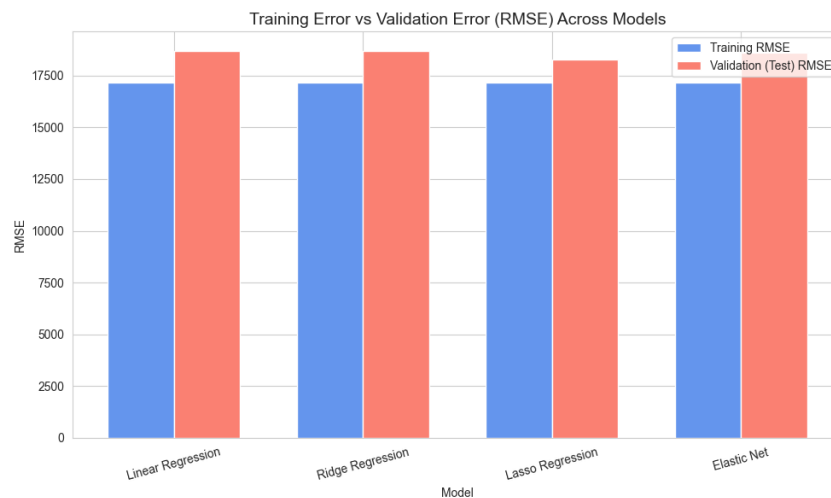


Figure 6: Training RMSE vs. Validation (Test) RMSE across all four models.

Inference: All four models show very similar training RMSE ($\approx 17,178$), with Lasso showing the smallest gap to its validation RMSE (18,290.21, a gap of 1112.10) among the four models, while Linear Regression shows the largest gap (18,685.10 training-to-validation gap of 1507.17). This is discussed further in Section 14.

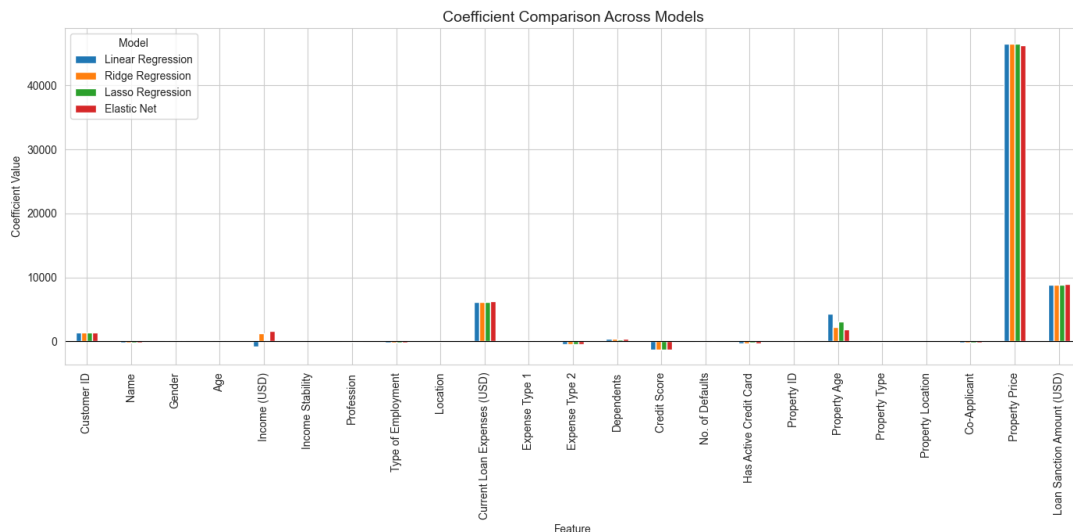


Figure 7: Coefficient values across all 23 features for Linear, Ridge, Lasso, and Elastic Net Regression.

Inference: **Property Price** has by far the largest-magnitude coefficient across all four models ($\approx 46,400$ – $46,550$), consistent with it being the feature most strongly correlated with the target. The coefficients for most other features are visually very similar across all four models, except **Income (USD)**, which is strongly positive under Ridge/Elastic Net, exactly zero under Lasso, and negative under plain Linear Regression — the clearest visual evidence in this experiment of coefficient instability being corrected by regularization.

12 Overfitting and Underfitting Analysis

Model	Training RMSE	Valid. RMSE	Gap (Val. – Train.)
Linear Regression	17177.9267	18685.1014	1507.1747
Ridge Regression	17177.9737	18667.4966	1489.5230
Lasso Regression	17178.1157	18290.2121	1112.0964
Elastic Net Regression	17178.4316	18619.8415	1441.4099

Difference between training and validation errors: All four models show a consistent gap between training RMSE ($\approx 17,178$) and validation/test RMSE ($\approx 18,290$ – $18,685$), indicating a mild degree of overfitting in every case — each model fits the training data somewhat better than it generalizes to unseen data, though the gap is modest relative to the overall RMSE magnitude (roughly 6–9% of the training RMSE).

Effect of regularization strength: Lasso, at its tuned $\alpha = 10$, achieved the smallest train–validation gap (1112.10) of all four models, indicating that its L_1 penalty most effectively reduced overfitting for this dataset, primarily by zeroing out the weakly-informative

Income (USD) feature. Ridge, despite also being tuned to $\alpha = 10$, produced only a marginally smaller gap than plain Linear Regression (1489.52 vs. 1507.17), suggesting that L_2 shrinkage alone had a comparatively small regularizing effect on this dataset. Elastic Net’s gap (1441.41) falls between Ridge’s and Lasso’s, consistent with its high `l1_ratio` = 0.8 blending mostly Lasso-like behaviour with a small Ridge contribution.

Improvement in generalization after tuning: All three regularized models were tuned via 5-fold cross-validated Grid Search rather than using their default $\alpha = 1$ (Ridge, Lasso) or $\alpha = 1$, `l1_ratio` = 0.5 (Elastic Net) settings. Tuning noticeably benefited Lasso in particular, whose training-time also increased from 0.0997s–1.9474s (default, untuned run in Section 15) to 1.5978s at the optimal $\alpha = 10$, alongside its improved generalization gap. No model in this experiment showed strong overfitting (i.e. a very small training error with a much larger validation error) or strong underfitting (i.e. both training and validation error being poor); all four sit in a similar, mildly-overfit regime typical of linear models applied to real-world tabular data with several dominant, well-behaved predictors.

13 Bias–Variance Analysis

Bias behaviour of Linear Regression: Linear Regression, having no regularization penalty, has the lowest bias of the four models — it is free to fit the training data as closely as ordinary least squares allows. This is reflected in it having a training RMSE (17177.93) essentially tied with the regularized models, but the largest train–validation gap (1507.17), consistent with low-bias models being more prone to capturing noise (higher variance) in the absence of a penalty term.

Variance reduction using Ridge and Elastic Net: Ridge’s L_2 penalty and Elastic Net’s blended penalty are both designed to shrink coefficient magnitudes and thereby reduce variance, trading a small amount of additional bias for improved stability. In this experiment, this trade-off is visible primarily in the **Income (USD)** coefficient, which swings from -860.06 (Linear Regression) to $+1227.48$ (Ridge) and $+1593.73$ (Elastic Net) — a sign of a poorly constrained, high-variance coefficient under plain Linear Regression that both regularized models pull toward a more stable range, at the cost of a non-zero bias being introduced. The overall reduction in the train–validation RMSE gap for Elastic Net (1441.41, versus Linear Regression’s 1507.17) is further evidence of a modest variance-reduction effect.

Feature sparsity effect in Lasso: Of the 23 features, Lasso’s L_1 penalty reduced exactly one coefficient (**Income (USD)**) to precisely zero at the tuned $\alpha = 10$, effectively removing it from the model. All other coefficients remained non-zero, though several (e.g. **Property Age**: $4361.36 \rightarrow 3060.28$, **Location**: $-19.63 \rightarrow -0.25$) were shrunk substantially in magnitude relative to Linear Regression. This limited sparsity (1 of 23 features eliminated) suggests that most of the 23 features carry at least some independent predictive signal for this target, with **Income (USD)** being the clearest exception — consistent with its very weak, near-zero correlation with the target observed in the EDA (Section 5, $r = 0.0460$).

14 Discussion

Observations: All four regression models achieved very similar performance (R^2 between 0.9001 and 0.9043), dominated by three strongly correlated monetary features (**Property**

Price, Loan Sanction Amount (USD), Current Loan Expenses (USD)). Regularization produced only modest improvements over plain Linear Regression in this experiment, with Lasso yielding the best overall test R^2 and the smallest overfitting gap, primarily by eliminating the uninformative `Income (USD)` feature. Ridge’s benefit over Linear Regression was comparatively small, and Elastic Net (which leaned heavily toward Lasso-like behaviour with `l1_ratio=0.8`) fell between the two.

Strengths: The experiment’s preprocessing pipeline correctly handled missing values (imputation rather than row deletion, preserving all 30,000 samples), consistently encoded categorical variables, and standardized features appropriately for regularized models. The hyperparameter tuning via 5-fold cross-validated Grid Search was performed correctly for all three regularized models, and all six required visualizations (target distribution, feature-vs-target scatter, predicted-vs-actual, residual plots, training-vs-validation error, and coefficient comparison) were generated successfully.

Limitations: As noted in Section 6, the ID-column removal logic did not match the dataset’s actual column-naming convention (space rather than underscore before “ID”), so `Customer ID` and `Property ID` were retained as model features; their negligible correlation with the target limited the practical impact of this, but it remains a preprocessing oversight worth correcting. Separately, the automatic target-column detection selected `Loan Amount Request (USD)` rather than `Loan Sanction Amount (USD)` (the quantity the assignment brief describes as “the loan amount sanctioned”), which also meant the sanctioned amount was available to the models as an input feature — worth being aware of, since a feature this closely related to the requested amount likely contributes to the already-high R^2 scores. Cross-validated MAE/MSE/RMSE were not tracked separately from the final test-set metrics, since `GridSearchCV` was scored on R^2 only (Table 2).

Improvements: Correcting the ID-column detection logic (e.g. matching on whitespace-separated “ID” as well as underscore-separated “_id”) and re-confirming which loan-amount column is the intended target with the dataset documentation would strengthen the preprocessing pipeline. Expanding `GridSearchCV` to score on multiple metrics (e.g. `scoring=['r2', 'neg_mean_absolute_error']` with `refit='r2'`) would allow Table 2 to be fully populated with cross-validated MAE/MSE/RMSE rather than only R^2 .

15 Conclusion

This experiment implemented and compared Linear, Ridge, Lasso, and Elastic Net regression for predicting the requested loan amount from the loan-application dataset. After preprocessing (missing-value imputation, categorical encoding, and feature standardization) and exploratory analysis, all four models were trained, hyperparameter-tuned where applicable via 5-fold cross-validated Grid Search, and evaluated using MAE, MSE, RMSE, and R^2 on a held-out test set.

Best performing classifier: Lasso Regression ($\alpha = 10$) was the best-performing model on the held-out test set, with MAE = 11346.24, RMSE = 18290.21, and $R^2 = 0.9043$, narrowly outperforming Elastic Net, Ridge, and plain Linear Regression, which all clustered closely together around $R^2 \approx 0.900$.

Optimal hyperparameters: Ridge and Lasso both selected the largest tested α (10), and Elastic Net selected the smallest tested α (0.01) with a high `l1_ratio` (0.8) — together suggesting that a moderate amount of L_1 -leaning regularization was most beneficial for this dataset, primarily through the removal of the single uninformative

Income (USD) feature rather than broad-based shrinkage.

Trade-off between accuracy and model complexity: The experiment shows that, for this dataset, going from an unregularized model (Linear Regression) to a lightly regularized one (Lasso) produced only a small accuracy improvement (R^2 : 0.9001 $\rightarrow$ 0.9043) but a proportionally larger improvement in generalization (train-validation RMSE gap: 1507.17 $\rightarrow$ 1112.10), at the cost of a substantially longer training time (0.0131s $\rightarrow$ 1.5978s). Since the target is overwhelmingly explained by a small number of strongly correlated features, the added complexity of tuning multiple regularized models yielded diminishing returns beyond what Linear Regression already captured — a useful, data-driven illustration of when regularization helps most (correcting individual unstable/uninformative coefficients) versus when it cannot substantially improve on a well-conditioned linear fit.

16 References

- [1] Scikit-learn Developers, “Linear Models — scikit-learn Documentation,” [Online]. Available: https://scikit-learn.org/stable/modules/linear_model.html.
- [2] Scikit-learn Developers, “Tuning the hyper-parameters of an estimator — scikit-learn Documentation,” [Online]. Available: https://scikit-learn.org/stable/modules/grid_search.html.
- [3] Kaggle, “Predict Loan Amount Data,” [Online]. Available: <https://www.kaggle.com/datasets>.